

IW AI Core Platform — Architecture Diagram

Type: Diagram Version: v10 Status: Published Generated: 2026-05-08 23:38:15

IW AI Core Platform — Architecture Diagrams

These diagrams cover the IW AI Core platform's structure, flows, and internal component relationships. Each section includes a 'Why' paragraph explaining when to use it and what insights it provides.

Diagram 1: System Architecture Overview

Why this diagram? Shows which physical components exist, how the iw-ai-core platform relates to managed projects, and where each type of traffic (human UI, agent CLI, daemon orchestration) flows. Use this when onboarding new engineers or troubleshooting connectivity issues between components.

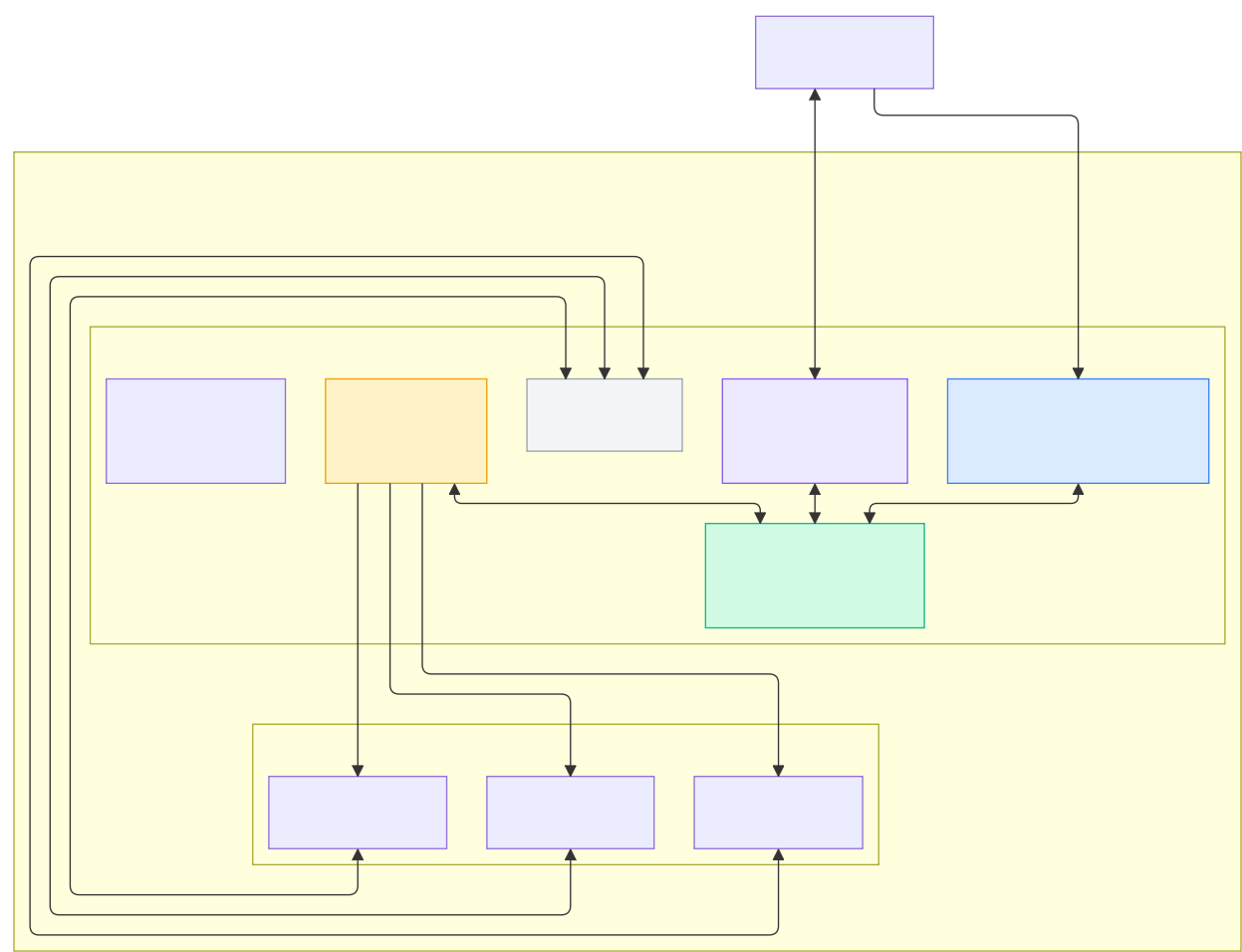


Figure 1: System Architecture Overview

Diagram 2: End-to-End Work Item Lifecycle

Why this diagram? Traces the exact sequence from typing `/iw-new-incident` to a merged and archived work item across all 5 phases. Use it to understand how a work item moves from draft through execution to completion, or to debug where a work item got stuck.

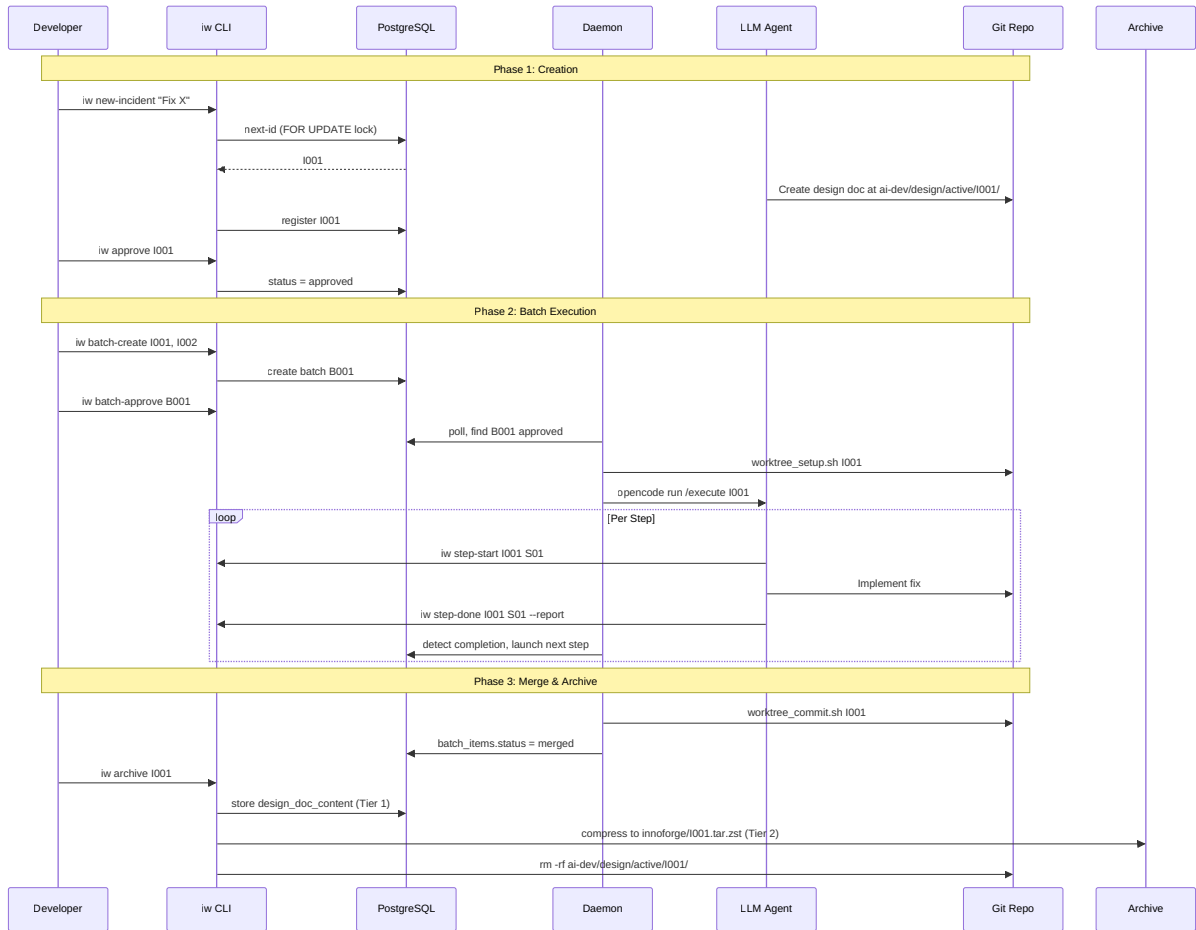


Figure 2: End-to-End Work Item Lifecycle

Diagram 3: Daemon Poll Cycle — Decision Tree

Why this diagram? Shows the daemon's poll loop decision tree: when it picks up a batch, how it monitors running agents, when it triggers timeout or stall, and how it proceeds to merge and archive. Use it to understand daemon behavior or to trace why the daemon took a particular action during a failure.

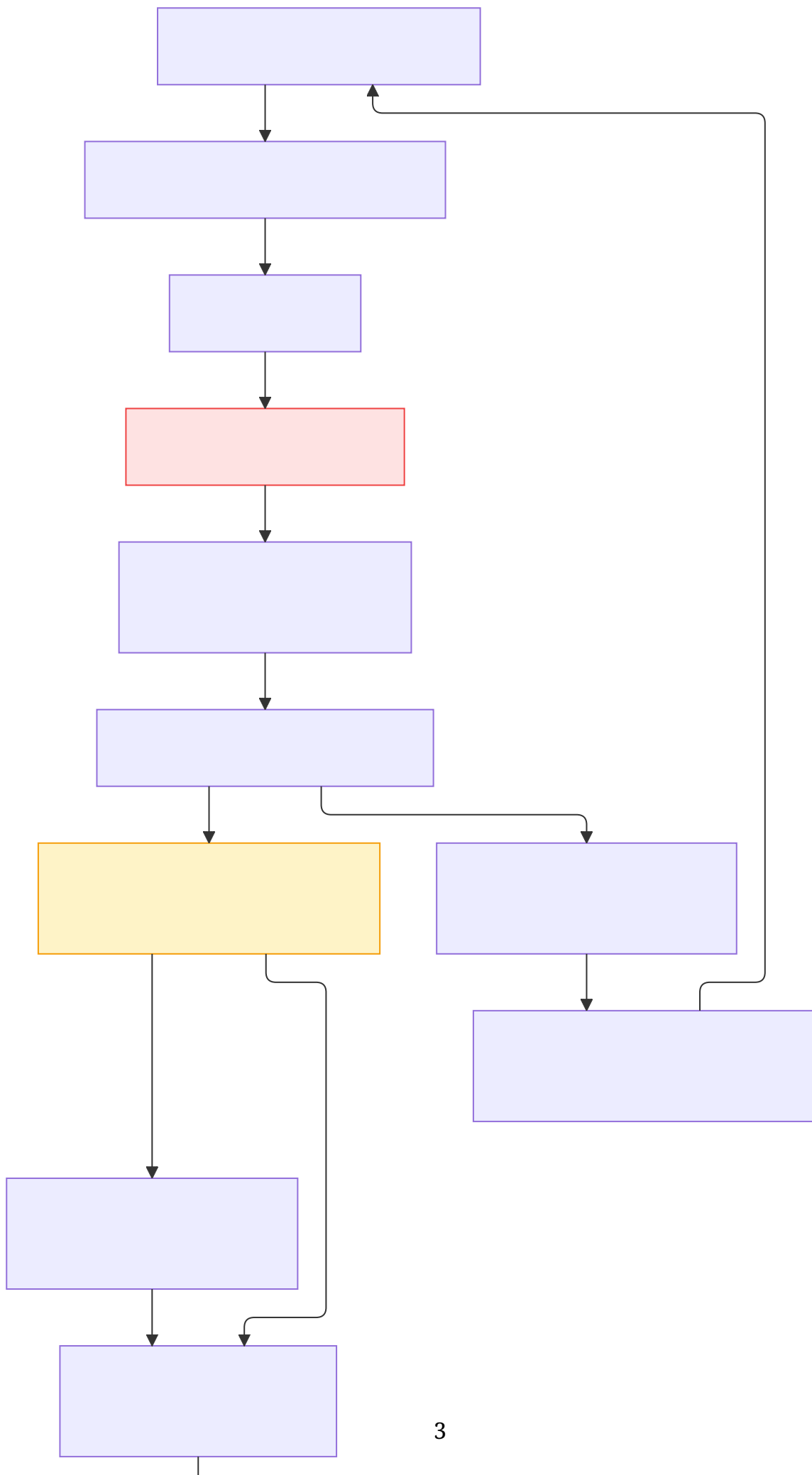


Diagram 4: Multi-Project Topology

Why this diagram? Shows how a single PostgreSQL instance, daemon, and dashboard serve multiple isolated projects simultaneously with worktrees and design files kept strictly within each project's own repository. Use it when understanding project isolation guarantees or debugging cross-project contamination.

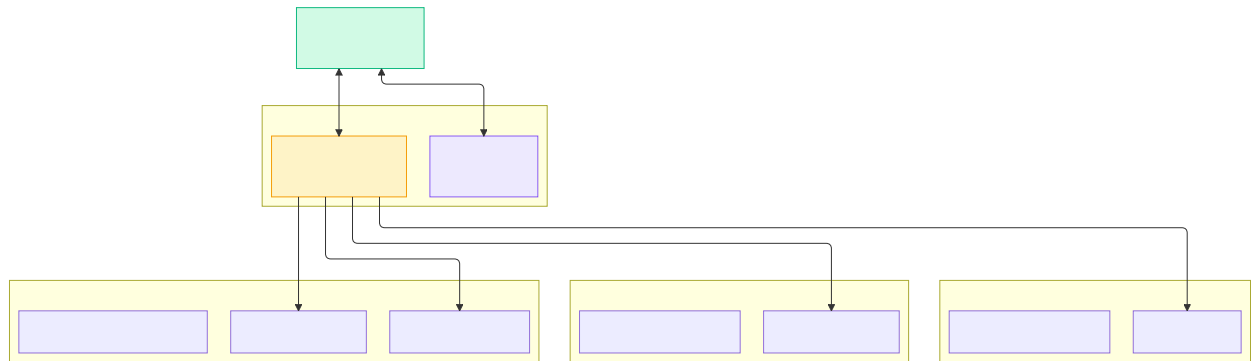


Figure 4: Multi-Project Topology

Diagram 5: Database Entity Relationships (Core Tables)

Why this diagram? Shows how the core tables relate to each other, which tables use composite `project_id` keys for multi-project isolation, and how work items flow from sequences through steps and `step_runs` to batches. Use it when designing new features against the schema or debugging data inconsistencies.

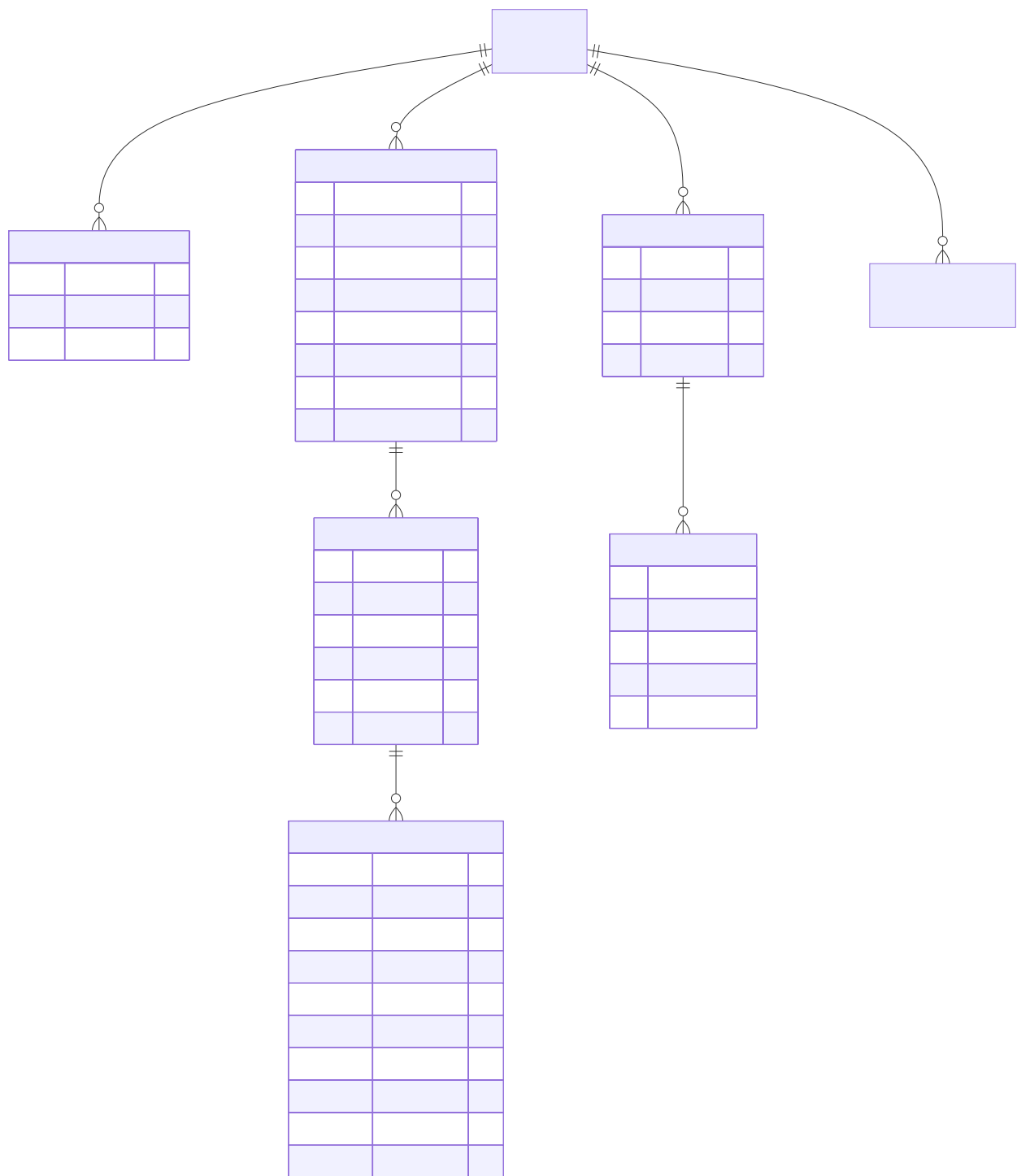


Figure 5: Database Entity Relationships (Core Tables)

Diagram 6: Component Architecture — Python Package

Why this diagram? Shows the internal structure of the orch/ Python package and how the CLI, daemon, database, and archive modules relate. Use it when adding new commands, modifying daemon behavior, or understanding where to place new code.

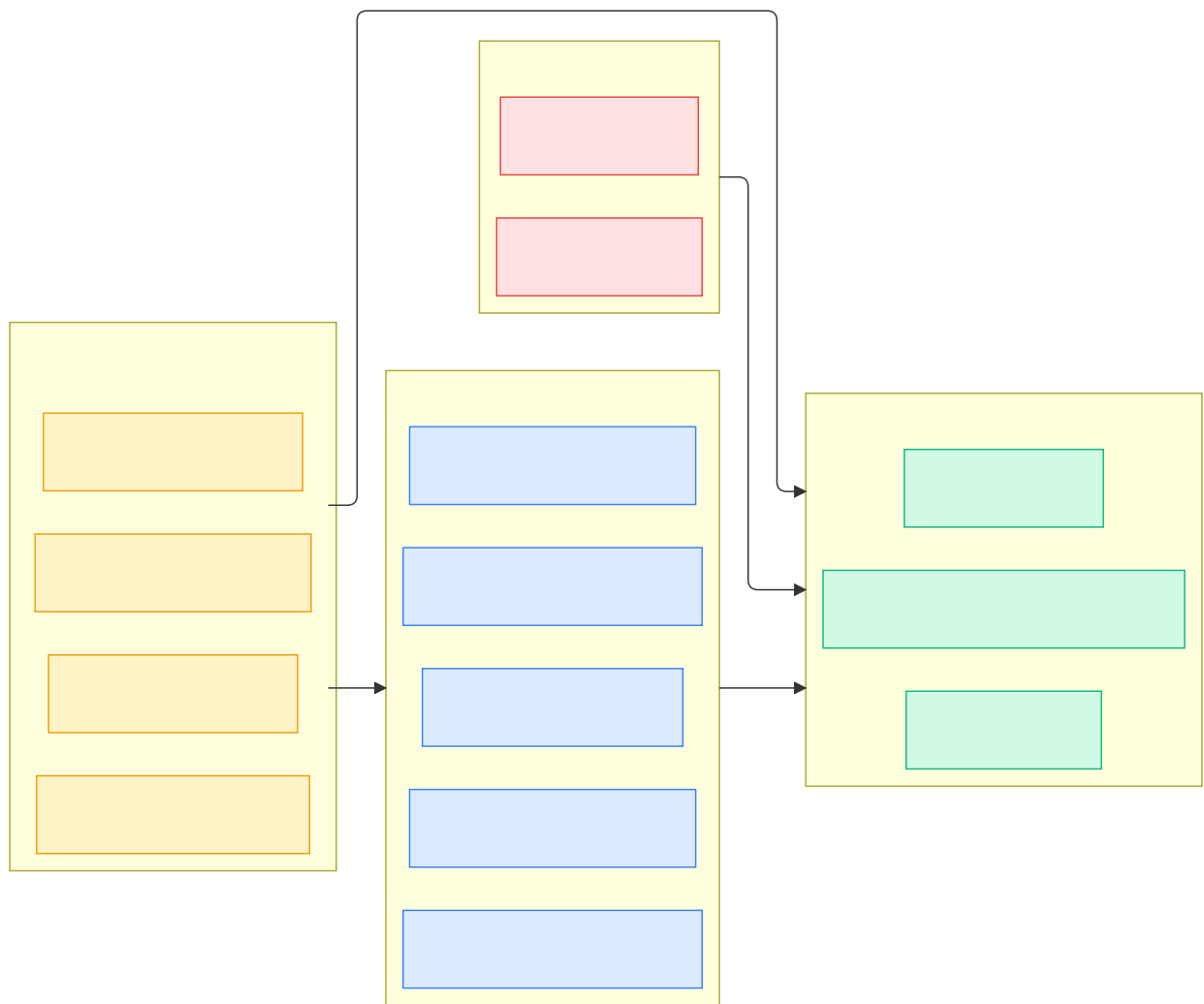


Figure 6: Component Architecture — Python Package

Diagram 7: Pre-Merge Migration Pipeline (CR-00021)

Why this diagram? Shows the 4-step migration pipeline from Phase 0 (rebase) through Phase 3 (rollback), illustrating how the daemon handles migration rebasing before merge to prevent `MultipleHeadsError`. Use it when debugging migration failures or understanding the CR-00021 rebase mechanism.

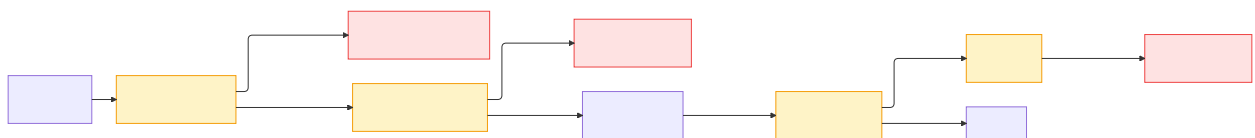


Figure 7: Pre-Merge Migration Pipeline (CR-00021)

Diagram 8: Worktree Container Lifecycle (F-00062)

Why this diagram? Shows when per-worktree Docker Compose stacks are created and torn down in relation to batch item state transitions. Use it when understanding worktree isolation or debugging container lifecycle issues.

